Reinforcement Learning

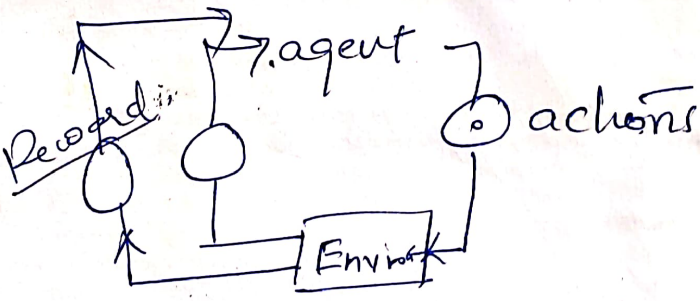
→ It is a learning that workson a feedback process, where AI agent explores the surroundings by hit and trial, taking action, learing from experience and improving its performance

Copyright © SciBulCom Ltd. 2025

ML

Study of comp. Alg for automatically constructing comp. s/w past exper & training data

→ Reinforcement problem can be formalized usg Markov Decesion process (MDP), agent continuously interacts with the environm and performs actions. At Each action the Environm responds & generates a neco state.



Reinforcement can be classified into 2 catego

Positive

increasing the tendancy that required behaviour occus again by adding Something

⇒ enhances the strength of the behaving agent truly impacts

negative

required occurs by avoiding negative

Applicati Reinforcement

Video games
Robotics
Text mng.

Scanned with OKEN Scanner

Advantage

- complex Real world problems
- Similar to learning of human
- Long Term Results.

Disadvantage

- not preferred for simple problems
- require huge data & computations
- Too much reinforcement leads to overload of states

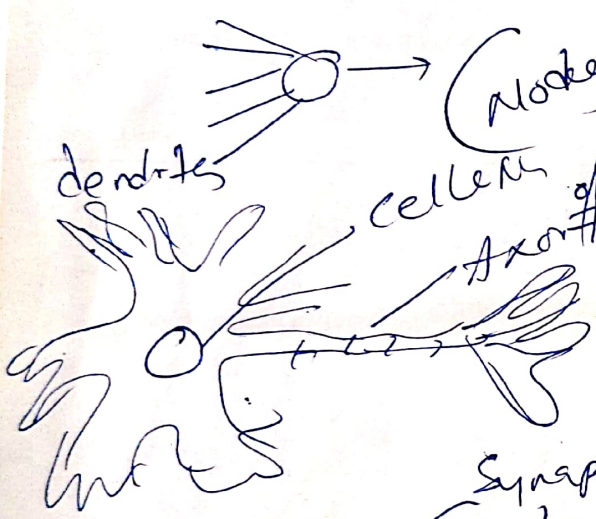
Scanned with OKEN Scanner

Brain & Neuron

→ Neurons – fundamental units of brain – receives i/p signals & perform an action based on the signals & sends o/p

→ ML., an artificial neuron is a mathematical function that mimics the behaviour of a biological neuron.

→ Takes multiple I/P → applies wt to it → Σ sums → passes result to an activation fn. → o/p



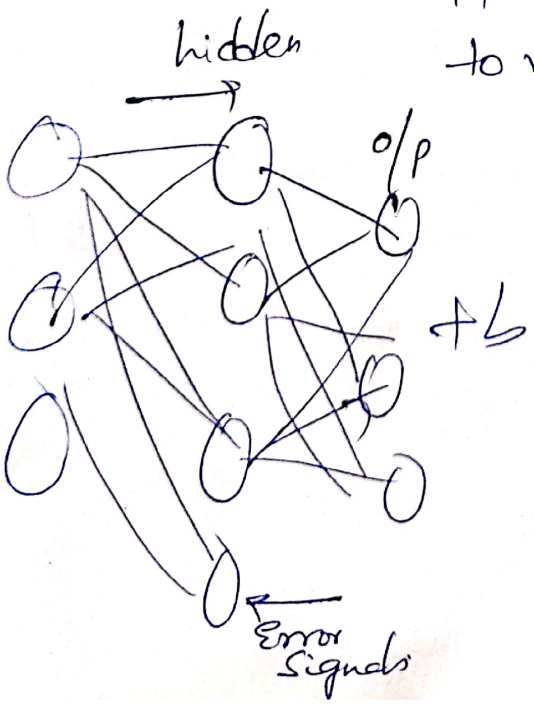
As neurons learn adjust their synaptic strength AN also adjust their weights through process called backpropogation

As neurons have synapses with varying strengths, AN have wts, associated w i/ps adjusted to minimize error

→ During learning it adjust its weight to minimize diff b/w predict & actual

As synaptic str N

bias



Scanned with OKEN Scanner

(2) Designing a Learning system in ML.

⇒ In ML, a computer is said to learn when — its performance improves with Experience while doing a specific Task

(9) Task (T) — what the system has to do

Performance measure (P) — how well, system is doing Task.

Experience (E) — Data / Examples from which the system learns

Def: "A computer program is said to learn from Experience (E) with respect to Task (T) and Performance measure (P) iff — the performance at Task T, measured by P, improves with Exp (E)

Eg Spam Email Detection

Task (T) ^ Spam, Notspam

Performance measure / how many e-mails correctly classified ⇒ (accuracy %)

Experience / set of e-mails already classified (labelled) as spam/notspam - used to train the system.

Steps in Designing a Learning system

1. Choosing Training Experience
2. Choosing the Target Function
3. Choosing Representation for Target function
4. Choosing the Learning (Function Approximation) algorithm
5. Final Design of the Learning system.

Choosing Training Experience — First imp. step is selecting training data

Training data = Experience given to ML algorithm

Good training data ⇒ good (better) model.

Poor training data ⇒ poor model

Scanned with OKEN Scanner

→ The Success / failure of ML model depends on

1. Type of feedback
   - Direct feedback
     ↳ correct answer is given (Supervised)
   - Indirect feedback
     ↳ only success / failure is known (reinforcement Learn)
2. Degree of feedback
   - Full feedback → Exact Labels
   - Partial feedback → only (reward / penalty)
3. Distribution of Examples
   - Data represent real world situations
   - Incomplete data ⇒ "wrong" Learning

Eg Chess

→ system tries different moves
→ If a move leads to winning ⇒ +ve feedback.
→ If it loses ⇒ it learns that it was bad move
overtime — chances of better moves increase (ie. Learing from Experience)

2. * control of Training Examples also imp ⇒ the learner can control "the sequence of training examples"

3. * Training examples need to be spread properly over different cases ie realistic & different examples
   ⇒ If ML learns only easy games, it may fail in real matches, so Training must include "different situations and difficulties"

CHOOSING TARGET FUNCTION

The Target function represents the correct output that the ML system must Learn, & tells what decision to make.

Eg Chess Target function = Nextmove
It decides which legal move to increase chances of winning.

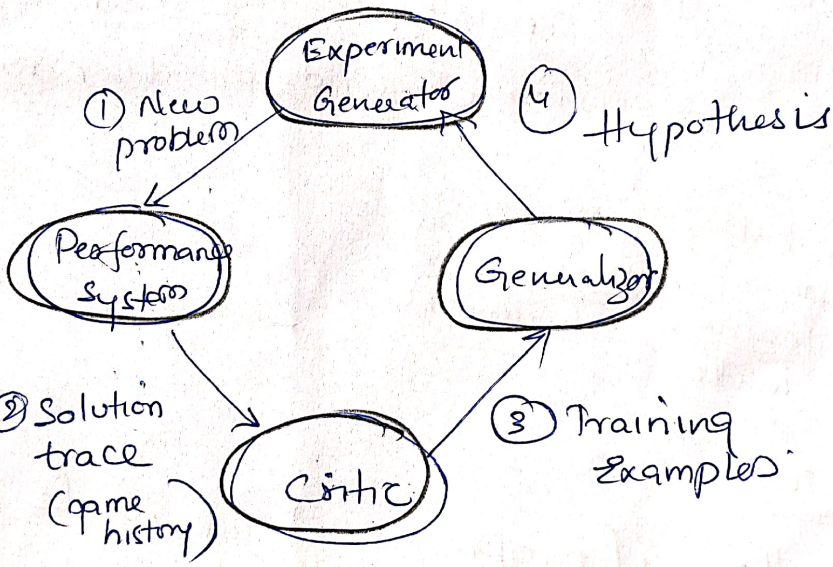
Scanned with OKEN Scanner

(3) CHOOSING REPRESENTATION FOR TARGET FUNCTION

The system must decide how to represent knowledge internally like Tables / Rules / Mathematical functions / Logical Expression

(4) CHOOSING THE LEARNING (FUNCTION APPROXIMATION) ALG.

→ Exact Target function is unknown
So, ML uses Approximation.
→ The Algorithm – Learns from examples
Tries moves
gets feedback (success/failure)
Improves step by step



(5) FINAL DESIGN OF Learning system

→ Final Learning system is designed using
(a) Training Examples (b) Performance measure
(c) feedback (d) correct / incorrect decision.

Eg Deep Blue chess computer – by IBM
" played against Garray Kasparov & won defeating human chess champion.
⇒ Power of Learning systems "

Scanned with OKEN Scanner

Perspectives and Issues of Machine Learning

Perspectives

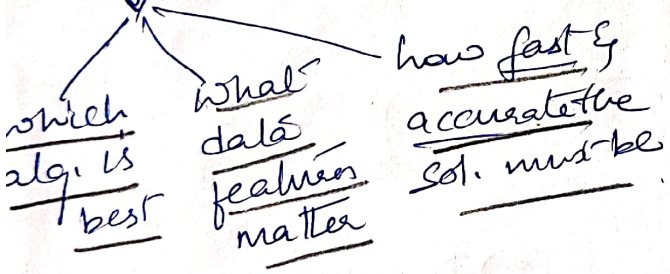
→ Perspective means how we look at / approach a problem in ML
→ ML Searches through a large number of possible solutions (hypotheses)
→ It selects the one that best fits the given data and prior knowledge.

Hypothesis in ML → is a possible explanation / solution
→ It is temporary and can change with more data
→ It can be true / false.

Eg: Sorting Eg Data can be sorted / unsorted

perspective ⇒ ML decides which algorithm to use based on datatype, size and requirement

Decision depends on User requirement, observation & perspective.



ISSUES

⇒ challenges / problems faced while building ML model

Issue 1: **Inadequate Training data.**

* ML needs large amount of data
Small / insufficient data ⇒ poor model / learning

Issue-2: **Data Quality Problems**

* Noisy data — data contains errors / irrelevant information & causes wrong predictions

Scanned with OKEN Scanner

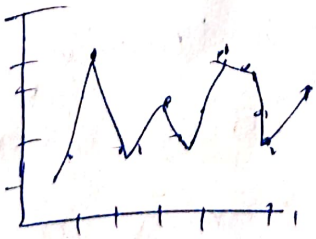
* Incorrect data — wrong labels / values ⇒ faulty models
* Poor Generalization — Model fails on new data

Issues :
* Poor Quality of Data is not desired.
Data must be — accurate, complete & clear
Poor quality ⇒ missing values, noisy data & Inconsistent
Result — Low accuracy and wrong classifications

Issue 3 : **Non-Representative Training data**
* Training data must represent real-world cases else model learns wrongly & predictions inaccurate
Eg: Training only day-time traffic, fails in night-traffic prediction.

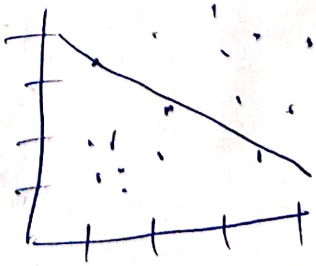
Issue 4 : **Overfitting** and **Underfitting**

overfitting
↓
Model learns too-much including noise.
- works well on training data, but fails on new data



Model tries to cover all data points in scatter plot & may look efficient, but not so good in reality

- Model learns too little
- cannot capture patterns
- Poor performance
reduced accuracy & unreliable predictions



Model unable to capture all the data points present in the plot.

Scanned with OKEN Scanner

ISSUE 5: **Monitoring and Maintenance**

* ML Models need continuous monitoring
* data keeps changing over time
* Model performance may degrade

Problems — data distribution changes
codes need updates
more resources are required.

"Getting bad Recommendations"
↓
Poor recommendations due to poor data, wrong model & bias in data

Lack of Skilled Resources
↓
ML needs skilled Professionals, but not enough — data scientists, ML Engineers

Wrong Segmentation Issues
↓
Poor targeting
Wrong business decisions
Eg: Same offer to diff. customer

Processing complexity
↓
computationally Expensive
Time Consuming

Data Bias: occurs when data favours certain outcomes
- Leads to unfair / inaccurate predictions
Eg - Training data collected only from one group.

** Remember — Before building an ML model, we must ask —

1. How much training data is sufficient?
2. Which kind of model be used?
3. Which learning method is suitable?
4. What type of data is required?

Scanned with OKEN Scanner

Concept Learning Task and Search

what is concept Learning.

↳ Task of learning a general rule (concept) from given training examples.

→ A concept is a boolean function that classifies as YES / NO.

→ Goal is to find a hypothesis that best fits the training examples.

Teaching a machine to Identify a rule that seperates YES Eg's from NO.

Concept Learning as a Search problem.

⇒ There are many possible hypotheses

→ We search through these hypotheses to find best one that matches all training Eg

→ If one hypotheses does not fit, we try another until we find the most suitable one

→ ∴ concept Learning as Search

Sky	AirTemp	Humidity	Wind	Water	Forecast	Enjoy Sport
Sunny	warm	Normal	Strong	warm	same	✓
Sunny	warm	high	"	warm	same	✓
Rainy	cold	high	"	warm	change	✗
Sunny	Warm	High	"	cold	change	✓

Scanned with OKEN Scanner

Hypothesis Space (Representation)

A hypothesis is written as a vector of constraints for each attribute

A specific value (Sunny)

? — any value acceptable

∅ — no value acceptable

< Sunny, warm, ?, strong, ?, ? >

Hypothesis Space (H) ⟹ set of all possible hypotheses.

very general by (accept almost)

very specific by (accept few)

distinct hypothesis

∅ ∅ ( ) — rejects all

< ?, ?, ?, ?, ?, ? > accepts all

	F_1	F_2
F_1 — A B	A	X
F_2 — X, Y	B	Y

16 possible ∅, ∅.

Instance Space

set of all possible i/p combinations

(A, X), (A, Y), (B, X), (B, Y).

? → accept anything

∅.

$3 \times 2 \times 2 \times 2 \times 2 \times 2 = 96$

Version Space = all hypotheses that are consistent with the training data

These hypotheses — Accept all the Examples

R: all — NO.

From many only a few remain valid

∴ Finds most consistent hypotheses.

Scanned with OKEN Scanner

FIND-S

→ concept learning algorithm used in supervised ML.

→ It goal - find the most specific hypothesis that fits all +ve training Egs.

* It ignores -ve and learns only from +ve

Why Find-S — Specific

Alg tries to find the most specific rule that classify all +ve egs

Steps

(1) start with the most specific hypothesis

$h \leftarrow \{\phi, \phi, \phi, \phi\}$

(2) Take 1st +ve Eg

(3) Replace ϕ with attribute value from the Eg

(4) For each +ve Eg ie if attribute match keep it else replace with ?

(5) Ignore all -ve Egs

(6) Final hypothesis is o/p

Attribute	Sky	Temp	Humidity	Wind	
	Sunny	Warm	Normal	Strong	✓
	S	Warm	High	Strong	✓
	Rainy	Cold	High	Strong	✗
	S	Warm	High	Weak	✓

Step 1: Initialize hypothesis

$h = \langle \phi, \phi, \phi, \phi \rangle$

Step 2: First +ve Eg

$\langle$ Sky warm, Normal, Strong $\rangle$

$h = \langle$ Sky, warm, normal, strong $\rangle$

Step 3: – Second +ve Eg.

$\langle$ Sky, warm, High, Strong $\rangle$

humidity diffn – ?

$\langle$ Sky, warm, ?, Strong $\rangle$

Step 4 – Ignore – no change

Step 5 – Next +ve $\langle$ Sunny, warm, high, heat $\rangle$

$\langle$ Sunny, warm, ?, ? $\rangle$ wind diffn. so

* Concept Learning - goal -

"Learn a rule that correctly classifies all training Eg's."
(hypothesis)

* There are many possible rules, so we define
Hypothesis Space (H) — all possible rules
Version Space (VS) — only those that match training data

* What Find-S is doing
- looks at +ve rules only and keep loosening (relaxing) the rule just enough to include them.

* Drawbacks of Find-S
- Ignores -ve examples
- Final hypothesis may still cover -ve cases
- Gives only one hypothesis
- Does not specify if others are possible.

* Why "Candidate Elimination"
- Real learning needs both YES & NO Eg's.
"what are ALL possible rules that agree with data"
⇓
leads to Version space & Candidate Elimination Algorithm

* Version Space - set of all hypothesis that classify (consistent) every training Eg correctly.
- Any one mistake ⇒ that hypothesis removed.

$$VS = \{ h \in H \mid h(x) = c(x) \text{ for all } x \text{ in } D \}$$

Scanned with OKEN Scanner

* Main Feature of Candidate Elimination
  - It does not store all hypotheses, instead stores boundaries.

Specific boundary
- Most Specific hypothesis
- covers only +ve Egs
- Nothing more specific than S is valid

Eg S = <Sunny, warm, ?>

General boundary
- Most general boundary
- Excludes -ve Egs
- Nothing more general than G is valid

Eg G = {
<Sunny, ?, ?>
<?, warm, ?>
<?, ?, weak>

Version Space

All hypotheses between S and G.

S ⊆ Version Space ⊆ G

How candidate Elimination (Works) Learns

For +ve Eg's – Remove hypotheses from G that does not hold
– Generalize S minimally

For -ve Eg's – Remove hypotheses from S that cover it
– Specialize G minimally

Positive S ↑ moves up

Negative G ↓ moves down.

Scanned with OKEN Scanner

Candidate Elimination Algorithm with Eg.

(1) Initialization — Assume 3 attributes (sky, temp, wind // Y/N)

Initialize S

$S = \langle \phi, \phi, \phi \rangle$

Initial G

$G = \langle ?, ?, ? \rangle$

Trainig Eg:

Sky	Temp	Wind	
Sunny	warm	strong	— Yes
Sunny	warm	weak	— Yes
Rainy	cold	strong	— No

(2) Eg1: (Sunny, warm, strong) → Yes

$S = \langle$ Sunny, warm, strong $\rangle$

G unchanged

(3) Eg 2 (Sunny, warm, weak) → Yes

$S = \langle$ Sunny, warm, ? $\rangle$ as strong / weak

G = unchanged.

(4) Eg 3 (Rainy, cold, strong) → No

current $G = \langle ?, ?, ? \rangle$

check G consistency with S

$S = \langle$ Sunny, warm, ? $\rangle$ ✓

$\{ \langle$ Sunny, ?, ? $\rangle$

$\langle$?, warm, ? $\rangle$

$\langle$?, ?, weak $\rangle \}$

(5) Final S & G

(6) Any hypothesis b/w S & G is valid

Scanned with OKEN Scanner

What is Linear discriminant

→ A method used to seperate data into classes using a straight line (or plane).

- It is mainly used in classification problems
- common method — Linear Discriminant Analysis

(goal is to find best straight line that seperates classes as clearly as possible)

Why LDA Imagine we have data points of 2 classes.

class1 : ○ ○ ○
class2 : • • •

} plotted in 2D.

Problem — No straight line seperate them in original space.

Sol: Project the data onto a new axis, where seperation becomes clear.

Main Idea of LDA —

Works on 2 conditions

- Maximize distance b/w classes
  - class1 mean - μ_1
  - class2 mean - μ_2
  - LDA say μ_1 & μ_2 far apart.
- Minimize Variance within class
  - *data points inside should be close to each other
  - avoids overlap

LDA

Step1: Start with labelled data

Eg: Face Images → Male / Female

Medical data → disease / No disease

Supervised Learning

Step 2: Compute class mean.

start & Find average point of each class.

Eg: Mean of class A

Mean of class B

Step 3: Measure Variance (spead)

how points spread within each class

LDA wants Low spread within class

Step 4: Find the best direction (new axis)

LDA finds a new axis

- class means far apart along that axis
- Points in the same class stay close

-This axis is -Linear Discriminant

Step 5: Project data onto new axis

Original - 2D, After Projection - 1D.

Scanned with OKEN Scanner

(R.. Frank Rosenblatt 1957)

+1.5 = 4.1
3) + 1.5 = 5.4

LDA — Is it Classification / Dimensionality Reduction

Both → Reduces dimensions

↓

Uses projection to classify data

2D → 1D.

(Reduce face image features before Classif.)

Reduce medical parameter before diagnosis

Real World Ex

Face Recognition

- Reduce no. of features
- Make classification faster & accurate

Medical diagnosis

- classify patients based on symptoms
- Reduce redundant health parameters

Customer Identific.

- seperate customer groups
- used in prediction system.

Linear discriminant Vs LDA

↓

A linear function used to seperate data points into different classes

A Supervised Learn that finds a linear combination of features to max. — class seperat min — intra class variance.

(Rule) Finds (Boundary) LDA

concept learn
Rule-based
+ves only
Most Specific hypoth
discrete features

Classification.
Mathematical.
+ve & -ve
Best seperating Line
Numeric features

Scanned with OKEN Scanner

Perceptron (By Frank Rosenblatt 1957)

* Basic building block of a neural network
* It acts as a simple decision maker.
* Perceptron is the first step to learn Machine Learning and deep learning
* Used mainly for binary classification [Y/N, Pass/Fail]
* A Supervised Learning algorithm for binary classification that works well for linearly separable data.

A Perceptron consists of

Inputs, Weights, Bias, Summation function, Activation Function, Learning Rule

Perceptron Working

1. Inputs are given ($x_1, x_2, x_3 \ldots x_n$)
2. Each input has a weight ($w_1, w_2, w_3 \ldots$)
3. Inputs are multiplied with weights.
4. All results are added together (Summation)
5. the result is passed to an Activation function.
6. output is generated (0 or 1)

Inputs ($x_1, x_2, \ldots x_n$)
Features / measurable attributes of a data point that perceptron uses to make a decision

Weights ($w_1, w_2 \ldots w_n$)
weights determine how strongly each input contributes to the prediction.
- weights are learned during Training, adjusting based on errors.

Scanned with OKEN Scanner

Bias — a constant value added to the weighted sum to shift the decision boundary
- allows perception to classify correctly even when all input features are zero.
- weights control how i/p influences the o/p Vs Bias controls when the perception activates independent of any input

x — y (predicted)

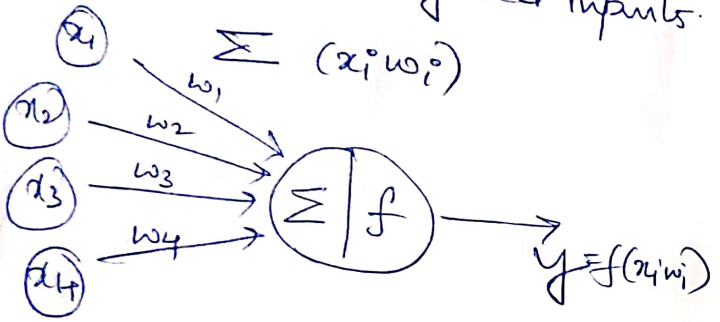
$y = 1.3 + 1.5$

Parts of a Perception (Node)

2 main parts

(a) Summation Unit

Adds all weighted inputs.



$\Sigma (x_i w_i)$

(b) Activation Function
- decides final output
- compares result with a threshold.
- If result $\geq$ threshold o/p = 1 else o/p = 0

$y = f(\Sigma x_i w_i)$

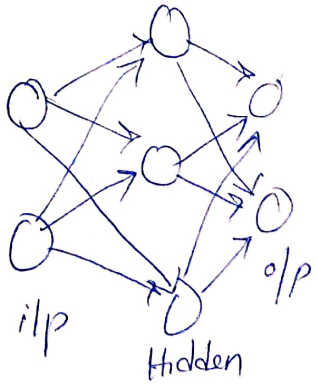
x — i/p
w — weight
f — activation function
y — o/p.

Activation Function

- takes the summed value
- produces output
- decides whether neuron should fire/not

ANN : Human brain — millions of neurons
Biological - Neurons

ANN has 3 Layers

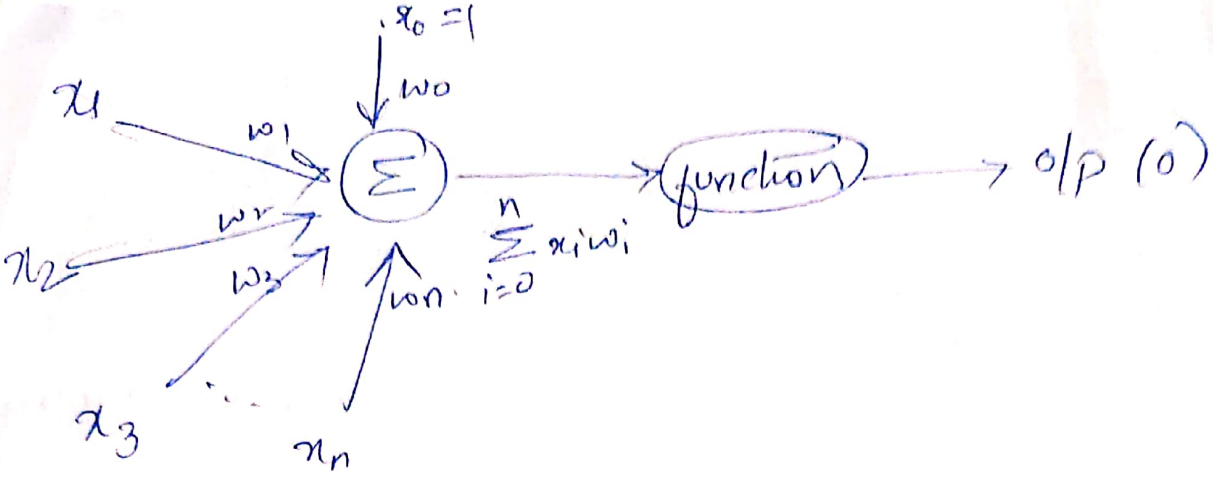


i/p Layer — receive i/p signals/info to hidden Layer.

hidden Layer — neurons are responsible for extracting info from the i/p, which is processed to generate o/p

o/p Layer — The generated output is responsible for producing and presenting o/p to user.

Scanned with OKEN Scanner



Initial weight $x_0 = 1$

In perception, we will give all the i/p's to the summation fn, it will add-up i/p & wts → sends to activation function fn, which generates o/p.

Here o - actual o/p
t - target o/p. (Expected)

if actual = Target ⇒ weights fixed
else ⇒ changed.

how to change weight

Pick weights randomly
keep applying iterations and check (o=t)
by applying delta rule (delta learning ⇒ Learning Rule)

$$w_i^o \leftarrow w_i^o + \Delta w_i^o$$

$$\Delta w_i^o = n\,(t-o)\,x_i^o$$

n = Learning rate (0.1, 0.2)
t = target o/p
o = actual o/p.
x_i = i/p of w_i^o

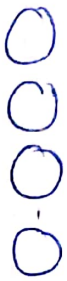
Scanned with OKEN Scanner

Types of Perceptron Models

Single layer perceptron [Straight line - 2 classes]

It consists of feed forward n/w and also includes a threshold transfer fn inside the Model

→ Analyzes linearly seperable objects with binary outcomes.



MultiLayer Perceptron

(Multiple lines) solves (non-linear problems)

not seperable by single line.

→ This node has the same structure but has greater no. of hidden layers.

→ It is also known as Back-propagation concept

2 stages

Forward stage

Backward stage

Scanned with OKEN Scanner

Linear Seperability

↓

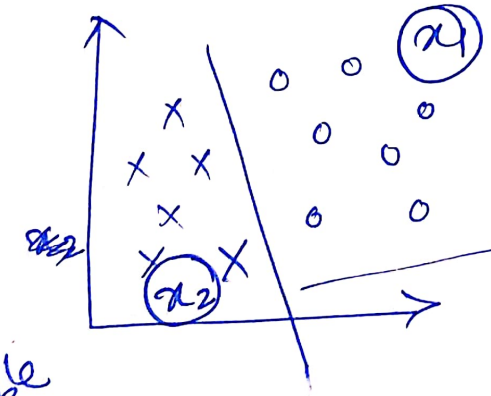
→ refers to the datapoints in binary classification problems, which can be seperated using linear decision boundary

→ A linear decision boundary is a line that divides data into classes

2D - Straight line 3D - plane n-D - hyper plane

* when 2 classes can be seperated by decision boundary represented by a linear Equation, — they are said to be linearly seperable

Linear seperability — refers that — classes of patterns with n dimensional vector $x = (x_1, x_2, x_3 \cdots x_n)$ can be seperated with a single decision surface (x_1, x_2)

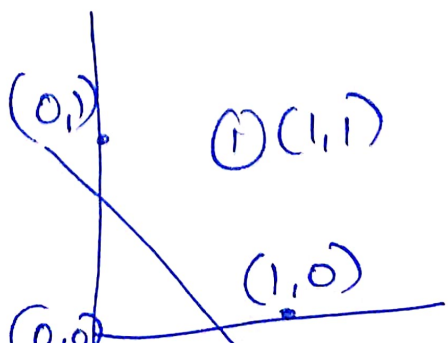


Hence patterns are linearly seperable

This line can be represented in linear Equation

Eg: Perceptron Eg for Linear Seperability of OR GATE

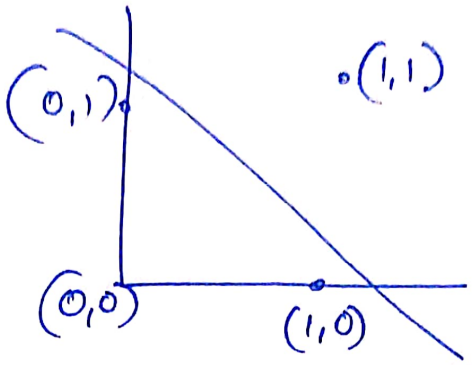
x_1	x_2	Y
0	0	0
0	1	1
1	0	1
1	1	1



Scanned with OKEN Scanner

Logical AND

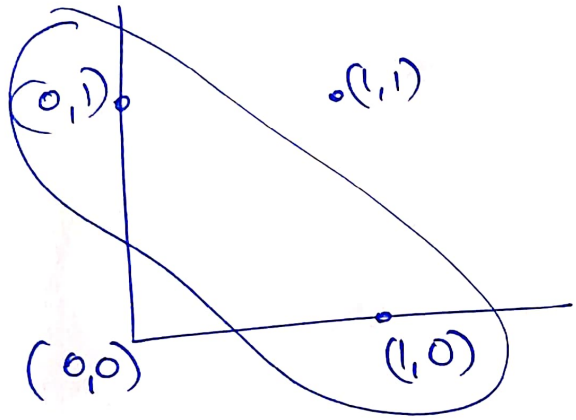
0	0	0
0	1	0
1	0	0
1	1	1



But for XOR.

Perceptron can not solve, we need to use a multi-layer perceptron

0	0	0
0	1	1
1	0	1
1	1	0



Mathematical

Input vector $X = (x_1, x_2 \dots x_n)$

classes linearly separable iff

$w_1x_1 + w_2x_2 \dots$

$w_nx_n + b = 0$

can seperate the classes.

Scanned with OKEN Scanner

Linear Regression

Used to predict — Salary, Sales, Age, Price

→ Most easiest and popular ML algorithm

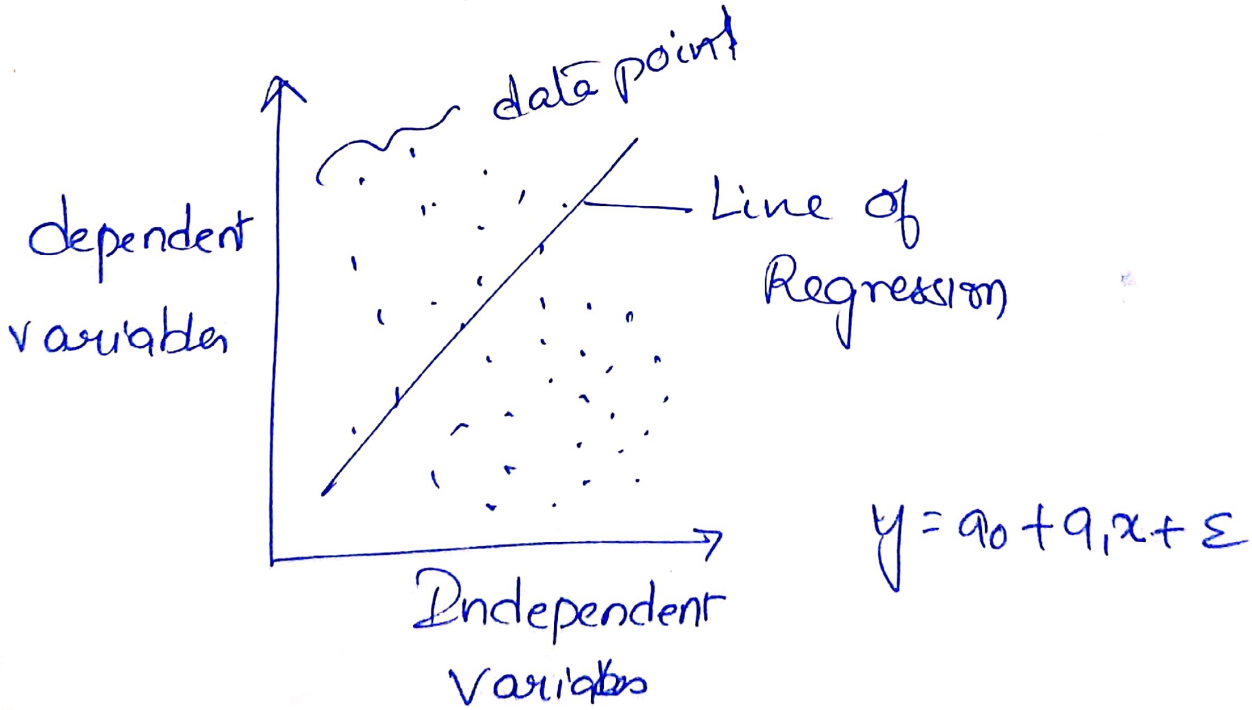
→ It is a statistical method that is used for predictive analysis

→ It makes predictions for continuous/real/numeric values.

→ It shows a linear relationship b/w a dependent (y) & one or more independent (x)

→ Linear Relationship

finds how the value of dependent variable is changing according to the value of independent variable



$y = a_0 + a_1 x + \varepsilon$

Scanned with OKEN Scanner

Mathematically, we represent a linear Regression

$$y = a_0 + a_1 x + \varepsilon$$

y - dependent variable
x - independent variable
a_0 - Incept of line (value of y when $x = 0$)
a_1 - Linear regression coefficient (how fast o/p changes based on i/p)

Higher coefficient → more impact on o/p.

ε - Random Error

Simple Linear Regression - uses only one independent variable

$y = mx + c$

y - dependent, x - Independent, m - Slope, c - Intercept

(how fast y changes with x)

Multi Linear Regression

$$y = a_0 + a_1 x_1 + a_2 x_2 + \cdots + a_n x_n$$

[salary depends on Exp, Edu, Skills]

Scanned with OKEN Scanner